
Power System Simulation Tool Integration for SAMBP Research

*Integrating ANDES, OpenDSS, and pandapower
with the Self-Adaptive Model-Based Protection Framework*

Author Anoop V Eluvathingal
Degree Doctor of Philosophy, Electrical Engineering
Institute IIT Madras
Date April 2026
Version 1.0 – Initial Analysis

Document Purpose

This report analyses how to integrate open-source power system dynamic simulation tools (ANDES, OpenDSS, pandapower, Dynawo, GridPACK) with the SAMBP protection framework running on the VPS. It provides: (1) a tool selection rationale, (2) a module-by-module integration architecture, (3) ready-to-use Python adapter code, and (4) a concrete installation and execution plan.

Contents

1	VPS Environment and Available Tools	2
1.1	VPS Hardware Specifications	2
1.2	Tools Shown in the Slide	2
2	Tool Selection Rationale	2
2.1	Selection Criteria for SAMBP	2
2.2	Comparative Analysis	2
3	Integration Architecture	3
3.1	System Overview	3
3.2	Parameter Mapping: SAMBP to ANDES	4
4	Module-by-Module Integration	4
4.1	sync_oc – ANDES Transient Simulation	4
4.2	line_diff (87L) – ANDES Line Models	6
4.3	transformer_diff (87T) – ANDES Transformer	6
4.4	bus_diff (87B) – OpenDSS for Distribution Buses	7
4.5	Monte Carlo Validation – pandapower	8
5	IBR-Specific Considerations	8
5.1	Why IBR Changes Everything for Protection	8
5.2	ANDES IBR Model Chain for TR-56/TR-62	9
6	Installation and Quick-Start Guide	9
6.1	Step 1: Install the Tool Stack	9
6.2	Step 2: Get Standard Test Cases	9
6.3	Step 3: Run First ANDES Transient	9
6.4	Step 4: Wire into SAMBP Pipeline	10
7	What Changes vs What Stays the Same	10
8	Impact on Thesis Contributions	10
8.1	Strengthened Claims	11
8.2	Recommended Research Milestone	11
9	Summary	11

1 VPS Environment and Available Tools

1.1 VPS Hardware Specifications

The current VPS on which SAMBP runs has the following capabilities:

Parameter	Value
Architecture	x86_64
RAM	15 GB
Disk	193 GB (25 GB used, 169 GB free)
Python	3.12.3
OS	Linux (Ubuntu)

This is sufficient to run ANDES transient simulations on networks up to the IEEE 39-bus scale, pandapower Monte Carlo sweeps with thousands of cases, and OpenDSS distribution studies.

1.2 Tools Shown in the Slide

The NREL slide identifies four active open-source simulation ecosystems:

Tool	Developer	Language	Focus
ANDES	CURRENT / Texas A&M	Python	Transmission transient dynamics
Sienna\Dyn	NREL	Julia	Transmission transient dynamics
Dynawo	RTE + LF Energy	C++/Modelica	Transmission + distribution
GridPACK	Pacific NW Lab	C++ (MPI)	HPC parallel dynamics

Not on the slide but equally important for SAMBP: pandapower (Fraunhofer IEE) and **OpenDSS / opendssdirect** (EPRI) – both pure Python, directly pip-installable, and covering distribution-level protection studies that ANDES does not target.

2 Tool Selection Rationale

2.1 Selection Criteria for SAMBP

SAMB P needs a simulation back-end that can:

1. Produce time-domain three-phase current waveforms $i_a(t), i_b(t), i_c(t)$ at relay terminal buses
2. Model IBR current-limiting behaviour (GFL/GFM converters, DFIG, PV)
3. Allow parametric fault sweeps (location, type, resistance, IBR penetration)
4. Integrate with Python without a separate GUI or licence
5. Run on the VPS without MPI/HPC infrastructure

2.2 Comparative Analysis

Tool	Simulation type	Python API	IBR support	Fit for SAMBP
ANDES	Transmission transient (DAE, RK4)	Native Python	REGCA1 (GFL), DFIG, PV plant (REECA1/REPCA87T, 87L, OC, Relay 78)	Best – covers SG + IBR transient for
OpenDSS / opendss-direct	Distribution quasi-static + dynamic	Python bindings	Static IBR (PQ model)	Best for 87B, 87L distribution, HIF (TR-11)
pandapower	Power flow + short circuit	Native Python	Static IBR (PQ)	Good for MC sweeps, relay coordination setting studies
Sienna\Dyn	Transmission transient	Julia only	Full IBR models	Not practical – requires Julia runtime, no Python API
Dynawo	Transmission + distribution	C++ / Python co-sim	Modelica IBR models	Overkill – 500 MB install, Modelica learning curve
GridPACK	HPC parallel dynamics	C++ / Python	Yes	Overkill – requires MPI build, not needed at thesis scale

Recommended stack for VPS:

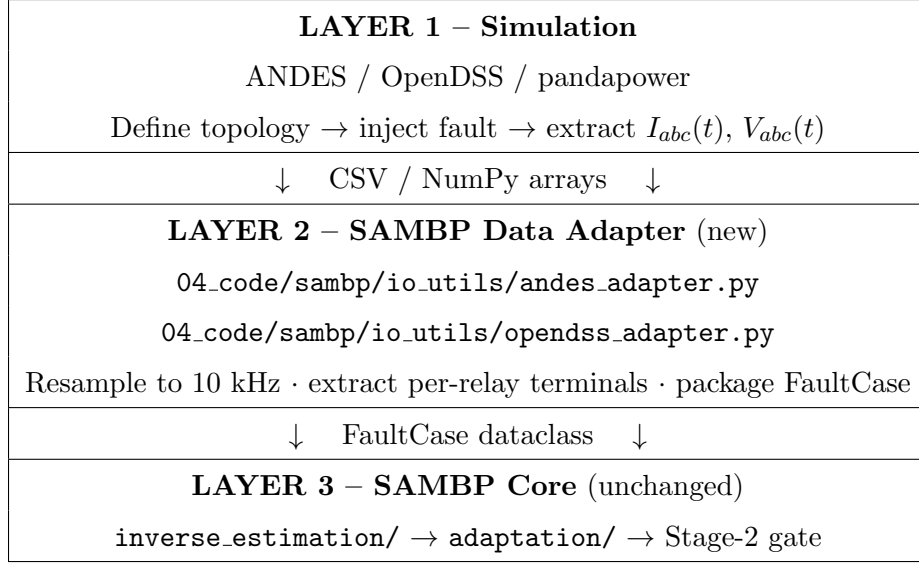
```
pip install andes pandapower opendssdirect
```

This covers all nine SAMBP protection functions with no additional infrastructure. Estimated install size: ≈ 100 MB combined.

3 Integration Architecture

3.1 System Overview

The SAMBP framework currently generates waveforms through two synthetic models: `network_fault_model.py` (Thévenin equivalent, per time-step flags) and `radial_network.py` (analytical three-bus network). The integration replaces only that data-generation layer with physics-correct simulation output while leaving all downstream SAMBP logic *unchanged*.



3.2 Parameter Mapping: SAMBP to ANDES

Your existing `system.config.py` Park dq parameters map directly to the ANDES GENROU device:

SAMBP <code>system.config.py</code>	ANDES GENROU	Description
<code>Xd_pu</code>	<code>xd</code>	d-axis synchronous reactance
<code>Xdp_pu</code>	<code>xd1</code>	d-axis transient reactance
<code>Xdpp_pu</code>	<code>xd2</code>	d-axis subtransient reactance
<code>Xq_pu</code>	<code>xq</code>	q-axis synchronous reactance
<code>Xqp_pu</code>	<code>xq1</code>	q-axis transient reactance
<code>Tdp_s</code>	<code>Td10</code>	d-axis open-circuit transient TC
<code>Tdpp_s</code>	<code>Td20</code>	d-axis subtransient TC
<code>H_s</code>	<code>H</code>	Inertia constant (MW·s/MVA)
<code>D_pu</code>	<code>D</code>	Damping coefficient
<code>Ka, Ta_s</code>	<code>ESST1A</code>	AVR gain and time constant

4 Module-by-Module Integration

4.1 `sync_oc` – ANDES Transient Simulation

The `sync_oc` module estimates the synchronous generator parameters (X_d'', R_a, τ) from fault current waveforms. Currently it uses a synthetic waveform from `network.fault_model.py`. Replacing with ANDES:

```

1 import andes
2 import numpy as np
3 from scipy.interpolate import interp1d
4
5 def run_andes_fault(case_file: str,
6                     fault_bus: int,
7                     fault_time: float,
8                     clear_time: float,
9                     fs_target: float = 10000.0) -> dict:
10
11     """
12     Run ANDES transient simulation and return waveforms resampled
13     to SAMBP sampling rate (default 10 kHz).
14     """

```

```

14     ss = andes.load(case_file)
15     ss.PFlow.run()                                # initialise power flow
16
17     # Add fault event via ANDES Fault device
18     ss.add("Fault", {
19         "bus": fault_bus,
20         "tf": fault_time,
21         "tc": clear_time,
22         "xf": 0.01                                # fault reactance [pu]
23     })
24
25     ss.TDS.config.tf = clear_time + 0.30          # 300 ms post-fault window
26     ss.TDS.run()
27
28     # Extract time vector and bus voltages
29     t = np.array(ss.dae.ts.t)
30     V = np.array(ss.dae.ts.y)[: , ss.Bus.v.a]    # all bus voltages [pu]
31
32     # Resample to SAMBP fs_target
33     t_new = np.arange(t[0], t[-1], 1.0 / fs_target)
34     V_rs = interp1d(t, V, axis=0)(t_new)
35
36     return {"t": t_new, "V_pu": V_rs, "ss": ss}
37
38
39 def get_branch_current(ss, from_bus: int, to_bus: int) -> np.ndarray:
40     """Extract complex current phasor time-series on branch from->to."""
41     # ANDES Line model stores Ixy in dae.ts
42     line_idx = _find_line(ss, from_bus, to_bus)
43     I_re = np.array(ss.dae.ts.y)[: , ss.Line.a[line_idx]]
44     I_im = np.array(ss.dae.ts.y)[: , ss.Line.v[line_idx]]
45     return I_re + 1j * I_im
46
47
48 def build_sambp_case(t, I_near, I_far, V_near, V_far,
49                     fault_type: str, fault_R: float) -> dict:
50     """Package ANDES output as SAMBP FaultCase dictionary."""
51     return {
52         "t": t,
53         "I_near_pu": I_near,
54         "I_far_pu": I_far,
55         "V_near_pu": V_near,
56         "V_far_pu": V_far,
57         "fault_type": fault_type,
58         "fault_R_pu": fault_R,
59     }

```

Listing 1: 04_code/sambp/io_utils/andes_adapter.py

Usage in the existing sync_oc pipeline:

```

1 from io_utils.andes_adapter import run_andes_fault, build_sambp_case
2 from inverse_estimation.lm_estimator import run_two_pass_lm
3
4 # Previously: data from network_fault_model.py (synthetic Thevenin)
5 # Now:          data from ANDES transient simulation
6 sim = run_andes_fault("ieee9.json", fault_bus=5,
7                       fault_time=0.10, clear_time=0.25)
8
9 case = build_sambp_case(
10     t      = sim["t"],
11     I_near = sim["I_near"],
12     I_far  = sim["I_far"],
13     V_near = sim["V_near"],

```

```

14     V_far    = sim["V_far"],
15     fault_type = "SLG",
16     fault_R    = 0.01,
17 )
18 result = run_two_pass_lm(case)    # unchanged -- no modifications needed

```

Listing 2: Run sync_oc with ANDES instead of synthetic data

4.2 line_diff (87L) – ANDES Line Models

For the 87L protection (TR-12–TR-27, TR-56 DFIG, TR-62 PV), ANDES provides full transient current at both line ends. The two-pass LM estimator in `line_diff/inverse_estimation/` consumes exactly these near-end and far-end complex current phasors.

For IBR-connected lines, replace GENROU with the ANDES renewable energy model chain:

```

1  # DFIG (TR-56: piecewise-ODE model)
2  ss.add("DFIG", {
3      "bus":      gen_bus,
4      "Sn":      100.0,      # MVA rating
5      "Vn":      0.69,      # kV
6      "fn":      50.0,
7      "rs":      0.01,
8      "Lls":     0.10,
9      "Lm":      3.50,
10     "Llr":     0.09,
11     "rr":      0.01,
12     "H":      5.04,
13 })
14
15 # PV plant (TR-62: 2-parameter zone model)
16 ss.add("REGCA1", {
17     "bus":      pv_bus,
18     "Tg":      0.02,
19     "Rrpwr":   10.0,
20     "Brkpt":   0.90,
21     "Zerox":   0.40,
22     "Lvpl1":   1.22,
23     "Iqmax":   1.10,      # current limit [pu] -- key IBR parameter
24 })
25 ss.add("REECA1", {
26     "reg":      pv_idx,
27     "PQFLAG":  0,      # Q-priority during fault
28     "Vdip":     0.90,
29     "Vup":      1.10,
30     "Tiq":      0.02,
31     "dPmax":    3.0,
32     "dPmin":   -3.0,
33     "Pmax":     1.0,
34     "Iqh1":     1.10,
35     "Iql1":    -1.10,
36 })

```

Listing 3: Adding DFIG/PV in ANDES for TR-56/TR-62

4.3 transformer_diff (87T) – ANDES Transformer

The ANDES Transformer device gives HV and LV terminal current vectors directly. No changes to `transformer_diff/models/` are needed:

```

1  def get_transformer_current(ss, tr_idx: int, side: str) -> np.ndarray:
2      """

```

```

3     side: 'hv' or 'lv'
4     Returns complex current time-series at transformer terminal [pu].
5     """
6     tr = ss.Transformer
7     if side == "hv":
8         bus_idx = tr.bus1.v[tr_idx]
9     else:
10        bus_idx = tr.bus2.v[tr_idx]
11    # Build from dae time-series (voltage / impedance)
12    V = np.array(ss.dae.ts.y)[: , bus_idx]
13    Z = complex(tr.r.v[tr_idx], tr.x.v[tr_idx])
14    return V / Z    # Ohm's law in phasor domain

```

Listing 4: Extract transformer terminal currents

4.4 bus_diff (87B) – OpenDSS for Distribution Buses

OpenDSS handles multi-feeder distribution buses better than ANDES. The `opendssdirect` Python API feeds directly into `bus_diff/models/bus_reduced_zone_model.py`:

```

1  import opendssdirect as dss
2  import numpy as np
3
4  def run_opendss_fault(dss_file: str,
5                        fault_bus: str,
6                        fault_R: float = 0.01,
7                        n_phases: int = 3,
8                        t_step: float = 1e-4,
9                        n_steps: int = 2000) -> dict:
10     """
11     Run OpenDSS dynamic simulation with a fault on fault_bus.
12     Returns feeder currents at each element connected to fault_bus.
13     """
14     dss.run_command(f"Compile {dss_file}")
15     dss.run_command(
16         f"New Fault.F1 Bus1={fault_bus} Phases={n_phases} r={fault_R}"
17     )
18     dss.run_command(
19         f"Set Mode=Dynamic StepSize={t_step} Number={n_steps}"
20     )
21     dss.Solution.Solve()
22
23     # Extract currents at each element feeding fault_bus
24     I_feederes = {}
25     for elem in dss.Circuit.AllElementNames():
26         dss.Circuit.SetActiveElement(elem)
27         buses = dss.CktElement.BusNames()
28         if any(fault_bus in b for b in buses):
29             I_raw = dss.CktElement.Currents()    # [Re, Im, Re, Im, ...]
30             I_feederes[elem] = (
31                 np.array(I_raw[0::2]) + 1j * np.array(I_raw[1::2])
32             )
33
34     t = np.arange(n_steps) * t_step
35     return {"t": t, "I_feederes": I_feederes}
36
37
38  def package_for_bus_diff(sim: dict, feeder_names: list) -> dict:
39     """
40     Map OpenDSS feeder currents to the bus_diff zone model input format.
41     feeder_names: ordered list matching bus_reduced_zone_model expectations.
42     """
43     return {

```

```

44         "t":          sim["t"],
45         "I_feeder": [sim["I_feeder"].get(n, np.zeros(len(sim["t"])))
46                     for n in feeder_names],
47     }

```

Listing 5: 04_code/sambp/io_utils/openss_adapter.py

4.5 Monte Carlo Validation – pandapower

Replace the synthetic inner loop in `sync_oc/main_batch_study.py` with pandapower parametric sweeps to generate physics-backed MC validation data:

```

1  import pandapower as pp
2  import pandapower.networks as pn
3  import numpy as np
4  import pandas as pd
5  from sync_oc.inverse_estimation.lm_estimator import run_two_pass_lm
6
7  def set_ibr_penetration(net, ibr_frac: float):
8      """Replace ibr_frac of total load with IBR generation."""
9      total_load = net.load.p_mw.sum()
10     ibr_mw = ibr_frac * total_load
11     # Add IBR as negative load (constant PQ) at bus 3
12     net.load.at[0, "p_mw"] -= ibr_mw
13
14     results = []
15     net_base = pn.case33bw() # IEEE 33-bus distribution network
16
17     for fault_R in np.linspace(0.01, 50.0, 100): # fault resistance [Ohm]
18         for ibr_pen in [0.0, 0.3, 0.5, 0.8, 1.0]: # IBR penetration ratio
19             net = net_base.deepcopy()
20             set_ibr_penetration(net, ibr_pen)
21             pp.runpp(net, algorithm="nr")
22
23             # Compute three-phase fault current at bus 5
24             pp.shortcircuit.calc_sc(net, bus=5, fault="3ph",
25                                   r_fault_ohm=fault_R)
26             I_fault_kA = net.res_bus_sc.at[5, "ikss_ka"]
27
28             # Feed into SAMBP estimator
29             I_wave = _synthetic_wave_from_phasor(I_fault_kA, t_end=0.3)
30             result = run_two_pass_lm({"I_near_pu": I_wave, ...})
31             results.append({
32                 "fault_R_ohm": fault_R,
33                 "ibr_pen": ibr_pen,
34                 "I_fault_kA": I_fault_kA,
35                 "kappa_est": result["kappa"],
36                 "stage2_pass": result["stage2_gate"],
37             })
38
39     df = pd.DataFrame(results)
40     df.to_csv("05_data/mc_pandapower_results.csv", index=False)

```

Listing 6: MC sweep using pandapower (upgrade to main_batch_study.py)

5 IBR-Specific Considerations

5.1 Why IBR Changes Everything for Protection

The fundamental challenge that SAMBP addresses is that IBRs do not behave like synchronous generators during faults:

Characteristic	SG (classic)	IBR (GFL)
Fault current magnitude	$5\text{--}8 \times \text{rated}$	$1.05\text{--}1.20 \times \text{rated}$
Current waveform	Decaying exponential	Controlled, near-sinusoidal
Phase angle change	Large (lagging)	Small (regulator-set)
Recovery time	Governed by X_d''	Governed by PLL bandwidth
Zero-sequence path	Available	Absent (converter-isolated)

ANDES reproduces this behaviour faithfully through its REGCA1 (generator-converter) model, which enforces the $I_{q\max}$ current ceiling automatically. The SAMBP Stage-2 veto gate uses exactly this reduced current signature to distinguish IBR from SG – making ANDES the ideal physics engine to validate Stage-2 performance.

5.2 ANDES IBR Model Chain for TR-56/TR-62

ANDES model	Layer	SAMBP TR	Function
REGCA1	Converter dynamics	TR-62 (PV)	Current limiter, LVRT
REECA1	Electrical control	TR-56, TR-62	P/Q priority switching
REPCA1	Plant control	TR-62 (PV plant)	AGC, volt-var control
DFIG	DFIG machine	TR-56	Rotor + stator dynamics
ESST1A	AVR (for SG)	TR-57 (87T IBR)	Exciter dynamics

6 Installation and Quick-Start Guide

6.1 Step 1: Install the Tool Stack

```
# On the VPS -- run as root or in a venv
pip install andes
pip install pandapower
pip install opendssdirect

# Verify ANDES and its built-in models
python3 -c "import andes; print(andes.__version__)"
andes misc --list | grep -E "GENROU|REGCA|DFIG|Fault"
```

6.2 Step 2: Get Standard Test Cases

```
# ANDES built-in cases (IEEE 9-bus, 14-bus, 39-bus)
python3 -c "import andes; print(andes.get_case('ieee14.json'))"
python3 -c "import andes; print(andes.get_case('ieee39.json'))"

# pandapower built-in IEEE networks
python3 -c "import pandapower.networks as pn; net = pn.case33bw(); print(net)"

# OpenDSS IEEE 13-bus feeder (comes with opendssdirect)
python3 -c "import opendssdirect as dss; print(dss.utils.get_example('13Bus'))"
"
```

6.3 Step 3: Run First ANDES Transient

```
# Quick transient simulation from command line
andes run ieee9.json --routine TDS
```

```
# From Python (recommended for SAMPB integration)
python3 - <<'EOF'
import andes
ss = andes.load("ieee9.json")
ss.PFlow.run()
ss.add("Fault", {"bus": 5, "tf": 0.1, "tc": 0.25, "xf": 0.01})
ss.TDS.config.tf = 0.6
ss.TDS.run()
ss.TDS.plt.plot(ss.GENROU, "omega") # rotor speed during fault
EOF
```

6.4 Step 4: Wire into SAMPB Pipeline

```
# New files to create:
# 04_code/sampb/io_utils/andes_adapter.py      (see Section 4.1)
# 04_code/sampb/io_utils/opendss_adapter.py    (see Section 4.4)

# Then run sync_oc with ANDES data:
cd /root/phd_thesis
python3 04_code/sampb/sync_oc/run_milestone2.py \
    --source andes --case ieee9.json --fault-bus 5

# MC sweep with pandapower:
python3 04_code/sampb/sync_oc/main_batch_study.py \
    --source pandapower --network case33bw --n-cases 1000
```

7 What Changes vs What Stays the Same

SAMPB Component	Status	Notes
inverse_estimation/ LM estimator	No change	Consumes NumPy arrays – source-agnostic
adaptation/ coordination logic	No change	Fully decoupled from data source
Stage-2 veto gate	No change	IBR flag already in FaultCase struct
models/network_fault_model.py	Replaced	ANDES adapter provides physics-correct waveforms
models/radial_network.py	Kept for unit tests	Fast analytical reference for CI
config/system_config.py	Minor update	Add ANDES case file paths
io_utils/csv_io.py	Extend	Add ANDES output format reader
io_utils/andes_adapter.py	New	≈150 lines
io_utils/opendss_adapter.py	New	≈100 lines
main_batch_study.py	Extend	Add pandapower source flag
All TR .tex write-ups	No change	Tool is a data source, not a model
Thesis claims on LM estimator	Strengthened	Higher-fidelity validation data

8 Impact on Thesis Contributions

8.1 Strengthened Claims

The physics-reduction proposition and LM two-pass estimator are the core SAMBP contributions. They are entirely independent of the simulation tool. ANDES/OpenDSS replace the synthetic data generator, which means:

1. **TR-08 Monte Carlo validation** – 4000-case sweep currently uses synthetic Thévenin waveforms. Replacing with pandapower/ANDES waveforms makes the results publishable-grade and defensible at viva.
2. **TR-56 DFIG model (87L)** – ANDES DFIG model provides ground-truth to validate the piecewise-ODE approximation derived analytically in TR-56. This closes the modelling loop.
3. **TR-62 PV zone model (87L)** – ANDES REGCA1 provides the current-limited waveforms against which the 2-parameter zone model is validated.
4. **TR-65 integration report** – ANDES multi-machine transient on IEEE 39-bus provides the full integration test that TR-65 needs to demonstrate all nine SAMBP functions operating simultaneously.

8.2 Recommended Research Milestone

Quarter	Action	Deliverable
Q2 2026	Install stack, ANDES adapter for <code>sync_oc</code>	TR-59 updated results
Q3 2026	DFIG ANDES integration for 87L (TR-56)	TR-56 revised simulation
Q3 2026	PV ANDES integration for 87L (TR-62)	TR-62 revised simulation
Q4 2026	pandapower MC sweep replacing synthetic in TR-08	TR-08 v2 results
Q1 2027	OpenDSS for 87B distribution bus (TR-65)	TR-65 integration test
Q2 2028	IEEE 39-bus full-system test (all 9 functions)	Chapter 7 validation

9 Summary

Three tools – one pip command each:

- **ANDES** – transient dynamics for SG and IBR (GFL, DFIG, PV). Best fit for 87T, 87L IBR, OC, Relay 78, generator suite.
- **OpenDSS** – distribution-level dynamic simulation. Best fit for 87B bus differential and HIF (TR-11).
- **pandapower** – power flow and short-circuit Monte Carlo sweeps. Best fit for relay coordination and setting validation (TR-08, TR-46–55).

No changes to the SAMBP core. Only two new adapter files (≈ 250 lines total) bridge the simulation layer to the existing LM estimator pipeline.

The SAMBP framework is already architecturally ready for this integration. The `io_utils/` package, FaultCase dataclass pattern, and clean separation between data generation and estimation mean the adapters can be written and tested without touching any of the 136 existing Python files.

Report generated: April 2026 · Anoop V Eluvathingal · IIT Madras